

Теоретические сведения.

Для чего JavaScript нужен в web-технологиях.

Язык JavaScript за короткое время смог стать одним из основных языков программирования для web. При помощи него можно как модернизировать существующий сайт на HTML, так и создать новое Web-приложение. Хотя JavaScript не настолько популярен как PHP, но, тем не менее, дает некоторые возможности, не свойственные для остальных языков программирования. К примеру, язык JavaScript позволяет совмещать разные web-технологии, причем эффективно.

Многие считают JavaScript клиентским языком, что действительно является правдой. В языке много внимания уделяется внедрению в HTML для бесперебойной работы в среде, которую поддерживают современные браузеры. Помимо этого, JavaScript является серверным языком сценариев. В этом плане первыми кто использовал JS были программные решения от Microsoft и Netscape.

Язык позволяет решать задачи разного уровня - его используют web-дизайнеры, Java-программисты, разработчики БД и обычные пользователи. Перед началом изучения JavaScript рекомендуется изучить базовые web-технологии. Навыки программирования приветствуются, но не являются необходимыми.

Для полного понимания JavaScript необходимо прояснить цель создания языка. Ведь его возможности уже давно перешагнули за пределы обычного интернет-браузера. Его используют на серверах, также он встроен в последние версии Windows. Этот язык является мощным инструментом для взаимодействия различных технологий и в итоге – создания продуманных и эффективных программных решений.

Необходимо понимать, что JavaScript не есть инструментом для разработки, он язык сценариев. Выполнение кода JavaScript является результатом совместной работы как браузера клиента, так и серверного ПО. В отличие от HTML, JavaScript полностью зависит от браузера, если последний не поддерживает его, код JavaScript не выполнится. В этом случае, код сценария даже может вывестись как текст. Так как JavaScript является языком сценариев, его можно использовать в довольно разных ситуациях. Но в большинстве случаев, основное применение приходится на клиентскую часть.

Первоначальную версию JavaScript разработала Netscape(еще до разработки Java), для улучшения HTML – на тот момент ему не хватало динамики. После появления языка Java, Netscape стала сотрудничать с компанией Sun(разработчик Java) для создания языка сценариев с синтаксисом похожим на Java. В результате этого сотрудничества и родился язык JavaScript. Он позволил четко разделить обязанности клиентской и серверной части. До этого серверная часть выполняла все операции, вплоть до контроля ввода данных, что было довольно неэффективно. Использование JavaScript значительно увеличило роль клиентской части, что действительно позволило создать клиент-серверную систему на его основе. Также, JavaScript дополнил возможности Java, который не позволял соединять несколько технологий на web-странице.

После появления в 1995г., язык JavaScript получил поддержку многих известных разработчиков программного обеспечения. При помощи их поддержки язык совершенствуется и используется не только в браузерах и в различных приложениях. Через некоторое время права на язык выкупила Microsoft, правда, получив права лишь на технологию. В результате вмешательства Microsoft был создан свой вариант JavaScript, который поддерживал Internet Explorer, но был несовместим с Netscape Navigator 3.0, который в свою очередь поддерживал свои версии JS. С этого момента началась эпоха несовместимых версий – одна и та же страница по-разному отображалась в разных браузерах. В результате этой борьбы Netscape ушла из рынка браузеров. Также, была разработана альтернатива JavaScript – VBScript для упрощения разработки на Visual Basic. Но тем не менее, именно JavaScript стал стандартом для Web.

Практическое применение технологии JavaScript.

Использование JavaScript в HTML

Конструкции языка JavaScript могут включаться в HTML документы двумя способами:

- как отмеченные тегом <SCRIPT> операторы и функции,
- как обработчики событий (event handlers), используя стандартные HTML теги.

SCRIPT теги:

Операторы языка (скрипты) с тегами SCRIPT используют формат:

```
<SCRIPT>
```

```
Операторы JavaScript ...
```

```
</SCRIPT>
```

Дополнительный атрибут LANGUAGE, определяющий язык программирования скрипта задается как:

```
<SCRIPT LANGUAGE="JavaScript"> или <SCRIPT LANGUAGE="LiveScript">
```

```
Операторы JavaScript ...
```

```
</SCRIPT>
```

Контейнер <SCRIPT> может включать произвольное число JavaScript операторов.

Пример. Простая скрипт-программа.

```

<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
document.write("Hello net.")
</SCRIPT>
</HEAD>
<BODY>
That's all, folks.
</BODY>
</HTML>

```

При выполнении скрипт-программы на экран будет выдано:
Hello net. That's all folks.

Скрытое кодирование

Скрипт-программы могут быть размещены в HTML документе внутри поля комментария. Этот прием используется для предотвращения интерпретации текста скрипта устаревшими WWW интерпретаторами (браузерами). Такие скрипты заключаются тегами комментария:

```

<!-- Begin to hide script contents from old browsers.
// End the hiding here. -->

```

Определение и вызов функций

Скрипты размещаются внутри SCRIPT тегов и выполняются после окончания загрузки HTML страницы. Определение функции - это формальное описание, задающее имя, структуру функции и последовательность операторов, реализующих определенный алгоритм. Вызов функции - это процедура передачи параметров и активизации функции для выполнения ее алгоритма.

Пример. Скрипт функции и комментарий.

```

<HEAD>
<SCRIPT LANGUAGE="JavaScript">
<!-- to hide script contents from old browsers
function square( i ) {
    document.write("The call passed ", i , " to the function.", "<BR>")
    return i * i
}
document.write("The function returned ", square(5), ".")
// end hiding contents from old browsers -->
</SCRIPT>
</HEAD>
<BODY>
<BR>
All done.
</BODY>

```

Скрипт-программа в вышеуказанном примере в результате выполнения выдает на экран:

```

The call passed 5 to the function.
The function returned 25.
All done.

```

В примере используется задание скрипт-программы внутри заголовка HEAD документа. Поскольку заголовок загружается и интерпретируется первым, тем самым гарантируется, что определение функции будет выполнено до первого к ней обращения.

Пример. Скрипт-программа с двумя функциями.

```

<HTML>
<HEAD>
<SCRIPT>
<!-- hide script from old browsers
function bar() {
    document.write("<HR ALIGN='left' WIDTH=25%>")
}
function output(head, level, string) {
    document.write("<H" + level + ">" + head + "</H" + level + "><P>" + string)
}
// end hiding from old browsers -->
</SCRIPT>
</HEAD>
<BODY>
<SCRIPT>
<!-- hide script from old browsers
document.write(bar(), output("Make Me Big", 3, "Make me ordinary. "))
// end hiding from old browsers -->
</SCRIPT>

```

```
<P>
Thanks.
</BODY>
</HTML>
```

Результат выполнения скрипта:

```
Make Me Big
Make Me ordinary.
Thanks.
```

Обработчики событий (Event Handlers) в скрипт-программе.

В приложениях программ на языке JavaScript широко применяются задание асинхронных операций, запрограммированных для выполнения реакции на определенные действия пользователя, в основном связанных с манипуляциями мышью. Эти процедуры вносят элементы интерактивности работы с HTML документами. и называются *обработчиками событий*.

Программы обработчиков событий пишутся на языке JavaScript и включаются в HTML документы в стандартных HTML тегах.

Общий формат синтаксиса:

```
<TAG eventHandler="JavaScript Code">
```

где:

TAG - стандартный HTML тег,

eventHandler - имя обработчика событий ,

JavaScript Code - текст программы обработчика событий.

Пример:

```
<INPUT TYPE="button" VALUE="Calculate" onClick="compute(this.form)">
```

Можно задавать произвольное число операторов языка в двойных кавычках в поле JavaScript Code. Если включается более одного оператора, они должны быть отделены символом (;).

Признаком хорошего тона в программировании обработчиков событий является задание их как функций, для возможного многократного к ним обращения из различных мест HTML документа.

Приведенный выше пример является частью HTML конструкции form, которая передается в качестве параметра функции compute(), реализующей алгоритм данного обработчика событий.

Тип события задан именем его обработчика onClick, и момент которого наступает при нажатии левой кнопки мыши на указанном поле типа "button" в конструкции form. Параметр this.form передает функции имя текущей form.

Пример. Скрипт программа с конструкциями form и обработчиками событий.

```
<HTML>
<!-- Start Example Code -->
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
function compute(form) {
    if (confirm("Are you sure?"))
        form.result.value = eval(form.expr.value)
    else
        alert("Please come back again.")
}
</SCRIPT>
</HEAD>
<BODY>
<FORM>
Enter an expression:
<INPUT TYPE="text" NAME="expr" SIZE=15 >
<INPUT TYPE="button" VALUE="Calculate" ONCLICK="compute(this.form)">
<BR>
Result:
<INPUT TYPE="text" NAME="result" SIZE=15 >
<BR>
</FORM>
</BODY>
<!-- End of Example Code -->
</HTML>
```

Выдача программы :

Enter an expression: _____ Calculate

Result: _____

Довольно часто при разработке сайта нам требуется **получать параметры** из запросов. Рассмотрим пример использования технологии JavaScript для получения GET параметров.

```
<script language="JavaScript" type="text/javascript">
function getQueryVariable() {
//получаем строку запроса (?a=123&b=qwe) и удаляем знак ?
var query = window.location.search.substring(1);
```

```

//получаем массив значений из строки запроса вида vars[0] = 'a=123';
var vars = query.split("&");
var arr = new Array();
//переводим массив vars в обычный ассоциативный массив
for (var i=0;i<vars.length;i++) {
    var pair = vars[i].split("=");
    arr[pair[0]] = pair[1];
}
return arr;
}

//пример использования
test = getQueryVariable();
for(i in test){
    document.write(i+'='+test[i]+'<br>');
}
</script>

```

Рассмотрим большой пример использования JavaScript.

Достаточно часто пользователям интернет приходится сталкиваться с большим объемом информации, представленным в виде таблицы. Не менее часто требуется получить результаты в ином порядке, чем они представлены первоначально. Большинство web-мастеров решает эту проблему применением сортировки на сервере, для чего используется перезагрузка страницы. Действительно, серверные языки предоставляют гораздо больше возможностей отсортировать многомерный массив по определенному значению, чем скриптовые языки, выполняющиеся непосредственно на стороне клиента. Решение сортировать данные на стороне сервера вполне оправданно с точки зрения трудозатрат программиста, так как многие серверные языки имеют встроенные функции сортировки многомерных массивов, и поэтому не требуется вдумываться в алгоритмы сортировки, что-то изобретать или перестраивать алгоритм под свои нужды. Но все-таки такое решение не оправдано, если подходить к этой проблеме с точки зрения пользователя. Вначале посетителю сайта требуется дождаться достаточно длительной загрузки страницы, просмотреть результаты, нажать на кнопку "Отсортировать" и... опять дожидаться, пока сервер закончит работу и вернет результат. Такие множественные перезагрузки страницы никак не способствуют популярности сайта у посетителя. В конце-концов, устав дожидаться очередной загрузки страницы, или испугавшись лишнего трафика, он покинет сайт в поисках более лояльного к нему web-мастера.

Начнем с функции fillArray, которая и будет эмулировать многомерный массив, а вернее создать класс-объект с членами - данными многомерного массива. Приведем ее код:

Листинг 1:

```

1  function fillArray( years, books, authors ) {
2      authors = upCs( authors, " " );
3      authors = upCs( authors, "-" );
4      books = upCs( books, " " );
5
6      this.years = years;
7      this.yweight = weight( years );
8      this.books = books;
9      this.bweight = weight( books );
10     this.authors = authors;
11     this.aweight = weight( authors );
12 }

```

Разберем данные по строкам. Во второй строке вызывается функция upCs с параметром authors и пробелом в качестве второго параметра. Функция upCs создает заглавные буквы в начале строки и перед каждым вхождением второго аргумента. Эта функция была написана исключительно потому, что не все обрабатывают данные должным образом и вполне могут написать имя писателя, например, с маленькой буквы. Применение такой функции устраняет возможную ошибку программиста в заполнении массива данными, а также позволяет быть уверенным, что в функции сортировки не будет неверного сравнения заглавных букв со строчными. Впрочем, если вы уверены в том, что данные будут занесены верно, можете убрать вызовы этой функции:

Листинг 2:

```

1  function upCs( str, param ) {
2      var tmpStr = str.substring( 0, 1 ).toUpperCase() + str.substring( 1 );
3      if( !param )
4          return tmpStr;
5      var separator = tmpStr.indexOf( param );

```

```

6   var retStr = tmpStr;
7   if( separator != -1 )
8     retStr = tmpStr.substring( 0, separator );
9
10  while( separator != -1 ) {
11    tmpStr = tmpStr.substr( separator + 1, 1 ).toUpperCase() + tmpStr.substring(
separator + 2 );
12    separator = tmpStr.indexOf( param );
13    if( separator != -1 )
14      retStr += param + tmpStr.substring( 0, separator );
15    else
16      retStr += param + tmpStr;
17  }
18
19  return retStr;
20 }

```

Эта же функция, вызванная с пустым вторым аргументом, возвращает результат с заглавной буквой только в начале строки. Теперь обратимся к строке 7, листинга 1. Здесь вызывается функция `weight`, которая возвращает числовые значения каждого символа аргумента в виде массива. Для чего ее нужно писать? Дело в том, что коды символов русского алфавита в браузере идут последовательно, кроме кода символа буквы "ё". Код этого символа больше кодов символов остального алфавита, поэтому приходится присваивать такое "весовое" значение этому символу, которое соответствовало бы его позиции в алфавите. Вот код этой функции:

Листинг 3:

```

1  function weight( str ) {
2    var retArray = new Array();
3
4    for( var i = 0; i < str.length; i++ ) {
5      var tmp = str.charCodeAt( i );
6      if( tmp >= 1046 && tmp < 1078 )
7        tmp++;
8      else if( tmp == 1025 )
9        tmp = 1046;
10     else if( tmp >= 1078 )
11       tmp++;
12     else if( tmp == 1105 )
13       tmp = 1078;
14     retArray[ i ] = tmp;
15   }
16
17   return retArray;
18 }

```

Ну что же, функции, необходимые для создания нашего многомерного массива готовы. Осталось создать массив и заполнить его данными:

Листинг 4:

```

1  var txt = [];
2  txt[ 0 ] = new fillArray( "1959", "фрейд", "сартр жан-поль" );
3  txt[ 1 ] = new fillArray( "1940", "подростки", "сэлинджер джером" );
4  txt[ 2 ] = new fillArray( "1946", "пена дней", "виан борис" );
5  txt[ 3 ] = new fillArray( "1948", "осадное положение", "каю альбер" );
6  txt[ 4 ] = new fillArray( "1899", "об иноческой жизни", "рильке райнер мария" );
7  txt[ 5 ] = new fillArray( "1849", "аннабель Ли", "по эдгар" );
8  txt[ 6 ] = new fillArray( "1917", "дагон", "лавкрафт говард" );
9  txt[ 7 ] = new fillArray( "1915", "процесс", "кафка франц" );
10 txt[ 8 ] = new fillArray( "1989", "египет Рамсесов", "монтэ пьер" );
11 txt[ 9 ] = new fillArray( "1932", "мастер и Маргарита", "булгаков михаил" );

```

Обратить внимание стоит лишь на то, что года, когда были написаны книги, передаются в функцию в качестве строкового параметра, а не числового. Делается это для того, чтобы не писать несколько функций сортировки.

Насколько быстр такой метод заполнения массива? На компьютере со средней производительностью загрузка 1000 элементов происходит за 450 - 800 миллисекунд. Но поскольку пользователь будет получать массив такого размера не сразу, а по частям, и размер этих частей будет сильно зависеть от скорости его соединения с интернет, то время, затрачиваемое на создание элемента массива и определение весовых значений его данных, будет незаметно пользователю.

Наконец, самая интересная часть, а именно - непосредственная работа с данными. В качестве функции сортировки данных используем сортировку Хоара, которая наиболее соответствует поставленной задаче. Вот ее код.

Листинг 5 :

```
1 function quickSort( l, h, type ) {
2   var low = l;
3   var high = h;
4   var rt = eval( "txt[ " + Math.round( ( l + h ) / 2 ) + " ]" );
5   var middle = new fillArray( rt.years, rt.books, rt.authors );
6
7   do {
8
9     while( isLow( eval( "txt[ " + low + " ]" ), middle, type ) )
10      low++;
11
12    while( isLow( middle, eval( "txt[ " + high + " ]" ), type ) )
13      high--;
14
15    if( low <= high ) {
16      var temp = txt[ low ];
17      txt[ low++ ] = txt[ high ];
18      txt[ high-- ] = temp;
19    }
20  } while( low <= high );
21
22  if( l < high )
23    quickSort( l, high, type );
24  if( low < h )
25    quickSort( low, h, type );
26 }
```

Перейдем к строкам 9 и 12. Здесь в качестве сравнения значений элементов вызывается функция isLow, которая принимает в качестве аргументов два объекта для сравнения и третий аргумент - имя члена класса, по которому производится сортировка. Приведем эту функцию сравнения:

Листинг 7 :

```
1 function isLow( low, high, type ) {
2   var len1 = low[ type ].length;
3   var len2 = high[ type ].length;
4   var length = len1 < len2 ? len1 : len2;
5
6   for( var i = 0; i < length; i++ ) {
7     var str1 = low[ type ][ i ];
8     var str2 = high[ type ][ i ];
9     if( str1 < str2 )
10      return true;
11     if( str1 > str2 )
12      return false;
13   }
14
15   if( len1 < len2 )
16     return true;
17   return false;
18 }
```

Здесь для обращения к членам класса используются квадратные скобки, как в массиве. Подробнее о таком способе работы с пользовательскими объектами можно прочитать у Дмитрия Котерова на <http://dklab.ru> в статье "Маленькие хитрости JavaScript". Следует обратить внимание на строки 7 - 8. Здесь используется необычный способ адресации

элемента. Предположим, что значение аргумента `type` равно `"yweight"`, то есть хотим отсортировать по годам и обращаемся к массиву-члену класса, который содержит данные о "весовом" значении строки с числом года. То есть фактически мы будем обращаться так: `low.yweight[ i ]`.

Рассмотрим функцию создания таблицы на основе DOM (Document Object Model), а также функции выбора сортировки. Вот код, который должен быть между тегами `body`:

Листинг 8 :

```
1 <table id="table">
2   <tr>
3     <td><a href="javascript:allocator( 'yweight' )">Год</a></td>
4     <td><a href="javascript:allocator( 'bweight' )">Книга</a></td>
5     <td><a href="javascript:allocator( 'aweight' )">Автор</a></td>
6   </tr>
7   <tbody id="tablebody"></tbody>
8 </table>
```

В таблице присутствует атрибут `id = "table"`, чтобы из функции можно было обратиться к этому узлу. Также присутствуют строка с именами колонок и вызовом функции выбора сортировки и `tbody` с `id = "tablebody"`. Зачем необходим `tbody`? Netscape Navigator 7.1 и вообще все браузеры на движке Gecko, а также Opera, не требуют этого элемента при динамическом создании таблицы. Но только вот этот элемент жизненно необходим, если вы хотите увидеть результат в браузере Internet Explorer. Если, создавая таблицу функциями DOM, вы пропустите создание такого элемента, то результата не увидите. Как создается таблица? Вот эта функция:

Листинг 9 :

```
1 function createTable( cStart, cType, cSize, cChange ) {
2   var tabbd = document.getElementById( "tablebody" );
3
4   if( !tabbd ) {
5     var table = document.getElementById( "table" );
6     var tbody = document.createElement( "tbody" );
7     tbody.id = "tablebody";
8     table.appendChild( tbody );
9     tabbd = document.getElementById( "tablebody" );
10  }
11
12  while( tabbd.hasChildNodes() ) {
13    var tmp = tabbd.childNodes[ 0 ];
14    tabbd.removeChild( tmp );
15  }
16
17  for( var counter = cStart; eval( counter + cType + cSize ); eval( "counter" +
18    cChange ) ) {
19    var tr = document.createElement( "tr" );
20
21    var td = document.createElement( "td" );
22    var tdtxt = document.createTextNode( txt[ counter ].years );
23    td.appendChild( tdtxt );
24    tr.appendChild( td );
25
26    td = document.createElement( "td" );
27    tdtxt = document.createTextNode( txt[ counter ].books );
28    td.appendChild( tdtxt );
29    tr.appendChild( td );
30
31    td = document.createElement( "td" );
32    tdtxt = document.createTextNode( txt[ counter ].authors );
33    td.appendChild( tdtxt );
34    tr.appendChild( td );
35    tabbd.appendChild( tr );
36  }
37 }
```

Итак, `createTable` принимает четыре аргумента. Первый из них (`cStart`) - определяет значение счетчика в цикле `for`. Второй (`cType`) - тип сравнения, то есть значения `">"`, `"<"` и т.д. Третий аргумент (`cSize`) определяет значение, при

сравнении с которым прекращается цикл. Четвертое же значение (cChange) может становиться инкрементом или декрементом, в зависимости от требования сортировки. Желательно подробно разобраться в этой строке кода, она достаточно важна. Без этих четырех аргументов пришлось бы писать несколько отдельных функций, которые бы различались только в этой строке. Этот прием достаточно удобный для общего применения, и я не исключаю возможности, что вы захотите его использовать в своих функциях.

Осталась последняя функции - allocator, которая, собственно, и управляет всеми сортировками и созданием таблицы.

Листинг 10 :

```

1  function allocator( arg ) {
2
3      if( flag == arg ) {
4          if( sorted == "up" ) {
5              createTable( 0, "<", txt.length, "++" );
6              sorted = "down";
7          } else {
8              createTable( txt.length - 1, ">=", 0, "--" );
9              sorted = "up";
10         }
11         return;
12     }
13
14     quickSort( 0, txt.length - 1, arg );
15     createTable( 0, "<", txt.length, "++" );
16     flag = arg;
17 }

```

Зачем сортировать повторно, если сортировка уже проводилась, а пользователь желает видеть то же самое, но в обратном порядке? Достаточно вывести значения массива в обратном порядке. Эта функция вызывает функцию сортировки только в том случае, если пользователь желает отсортировать по другому критерию. Для этого введены глобальные переменные - `flag` и `sorted`. Достаточно хорошо можно понять суть работы, если использовать какой-нибудь отладчик, например, `Microsoft Visual Inter Dev`.

Итак, если вы следили внимательно за создаваемым кодом, у вас должна получиться приблизительно такая страница:

```
<html>  
<head>  
  <title>Сортировка таблицы средствами JavaScript</title>  
  <meta http-equiv="Content-type" content="text/html; charset=Windows-1251">  
  <script language="JavaScript" type="text/javascript">  
    var txt = [];  
    var flag;  
    var sorted;  
  
    function upCs( str, param ) {  
      var tmpStr = str.substring( 0, 1 ).toUpperCase() + str.substring( 1 );  
      if( !param )  
        return tmpStr;  
      var separator = tmpStr.indexOf( param );  
      var retStr = tmpStr;  
      if( separator != -1 )  
        retStr = tmpStr.substring( 0, separator );  
  
      while( separator != -1 ) {  
        tmpStr = tmpStr.substr( separator + 1, 1 ).toUpperCase() + tmpStr.substring(  
separator + 2 );  
        separator = tmpStr.indexOf( param );  
        if( separator != -1 )  
          retStr += param + tmpStr.substring( 0, separator );  
        else  
          retStr += param + tmpStr;  
      }  
  
      return retStr;  
    }  
  </script>  
</head>
```

```

}

function weight( str ) {
    var retArray = [];

    for( var i = 0; i < str.length; i++ ) {
        var tmp = str.charCodeAt( i );
        if( tmp >= 1046 && tmp < 1078 )
            tmp++;
        else if( tmp == 1025 )
            tmp = 1046;
        else if( tmp >= 1078 )
            tmp++;
        else if( tmp == 1105 )
            tmp = 1078;
        retArray[ i ] = tmp;
    }

    return retArray;
}

function fillArray( years, books, authors ) {
    authors = upCs( authors, " " );
    authors = upCs( authors, "-" );
    books = upCs( books, " " );

    this.years = years;
    this.yweight = weight( years );
    this.books = books;
    this.bweight = weight( books );
    this.authors = authors;
    this.aweight = weight( authors );
}

function isLow( low, high, type ) {
    var len1 = low[ type ].length;
    var len2 = high[ type ].length;
    var length = len1 < len2 ? len1 : len2;

    for( var i = 0; i < length; i++ ) {
        var str1 = low[ type ][ i ];
        var str2 = high[ type ][ i ];
        if( str1 < str2 )
            return true;
        if( str1 > str2 )
            return false;
    }

    if( len1 < len2 )
        return true;
    return false;
}

function quickSort( l, h, type ) {
    var low = l;
    var high = h;
    var rt = eval( "txt[ " + Math.round( ( l + h ) / 2 ) + " ]" );
    var middle = new fillArray( rt.years, rt.books, rt.authors );

    do {

        while( isLow( eval( "txt[ " + low + " ]" ), middle, type ) )
            low++;

        while( isLow( middle, eval( "txt[ " + high + " ]" ), type ) )
            high--;

        if( low <= high ) {
            var temp = txt[ low ];
            txt[ low++ ] = txt[ high ];
            txt[ high-- ] = temp;
        }
    } while( low < high );
}

```

```

    }
    } while( low <= high );

    if( l < high )
        quickSort( l, high, type );
    if( low < h )
        quickSort( low, h, type );
}

txt[ 0 ] = new fillArray( "1959", "фрейд", "сартр жан-поль" );
txt[ 1 ] = new fillArray( "1940", "подростки", "сэлинджер джером" );
txt[ 2 ] = new fillArray( "1946", "пена дней", "виан борис" );
txt[ 3 ] = new fillArray( "1948", "осадное положение", "камю альбер" );
txt[ 4 ] = new fillArray( "1899", "об иномечской жизни", "рильке райнер мария" );
txt[ 5 ] = new fillArray( "1849", "аннабель Ли", "по эдгар" );
txt[ 6 ] = new fillArray( "1917", "дагон", "лавкрафт говард" );
txt[ 7 ] = new fillArray( "1915", "процесс", "кафка франц" );
txt[ 8 ] = new fillArray( "1989", "египет Рамсесов", "монте пьер" );
txt[ 9 ] = new fillArray( "1932", "мастер и Маргарита", "булгаков михаил" );

function createTable( cStart, cType, cSize, cChange ) {
    var tabbd = document.getElementById( "tablebody" );

    if( !tabbd ) {
        var table = document.getElementById( "table" );
        var tbody = document.createElement( "tbody" );
        tbody.id = "tablebody";
        table.appendChild( tbody );
        tabbd = document.getElementById( "tablebody" );
    }

    while( tabbd.hasChildNodes() ) {
        var tmp = tabbd.childNodes[ 0 ];
        tabbd.removeChild( tmp );
    }

    for( var counter = cStart; eval( counter + cType + cSize ); eval( "counter" +
cChange ) ) {
        var tr = document.createElement( "tr" );

        var td = document.createElement( "td" );
        var tdtxt = document.createTextNode( txt[ counter ].years );
        td.appendChild( tdtxt );
        tr.appendChild( td );

        td = document.createElement( "td" );
        tdtxt = document.createTextNode( txt[ counter ].books );
        td.appendChild( tdtxt );
        tr.appendChild( td );

        td = document.createElement( "td" );
        tdtxt = document.createTextNode( txt[ counter ].authors );
        td.appendChild( tdtxt );
        tr.appendChild( td );
        tabbd.appendChild( tr );
    }
}

function allocator( arg ) {

    if( flag == arg ) {
        if( sorted == "up" ) {
            createTable( 0, "<", txt.length, "++" );
            sorted = "down";
        } else {
            createTable( txt.length - 1, ">=", 0, "--" );
            sorted = "up";
        }
    }
    return;
}

```

```

        quickSort( 0, txt.length - 1, arg );
        createTable( 0, "<", txt.length, "++" );
        flag = arg;
    }

</script>
</head>
<body>

<table border="1" id="table">
    <tr>
        <td><a href="javascript:allocator( 'yweight' )" >Год</a></td>
        <td><a href="javascript:allocator( 'bweight' )" >Книга</a></td>
        <td><a href="javascript:allocator( 'aweight' )" >Автор</a></td>
    </tr>
    <tbody id="tablebody"></tbody>
</table>

<script language="JavaScript" type="text/javascript">
    createTable( 0, "<", txt.length, "++" );
</script>

</body>
</html>

```

Сохраните эту страницу в формате .html, откройте в браузере и вы увидите следующую таблицу:

Год	Книга	Автор
1959	Фрейд	Сартр Жан-Поль
1940	Подростки	Сэлинджер Джером
1946	Пена дней	Виан Борис
1948	Осадное положение	Камю Альбер
1899	Об иномечской жизни	Рильке Райнер Мария
1849	Аннабель Ли	По Эдгар
1917	Дагон	Лавкрафт Говард
1915	Процесс	Кафка Франц
1989	Египет Рамсесов	Монтэ Пьер
1932	Мастер и Маргарита	Булгаков Михаил

Нажмите на одну из ссылок в шапке таблицы и данные отсортируются по этому параметру. Например, сортировка по автору даст следующий результат:

Год	Книга	Автор
1932	Мастер и Маргарита	Булгаков Михаил
1946	Пена дней	Виан Борис
1948	Осадное положение	Камю Альбер
1915	Процесс	Кафка Франц
1917	Дагон	Лавкрафт Говард
1989	Египет Рамсесов	Монтэ Пьер
1849	Аннабель Ли	По Эдгар
1899	Об иномечской жизни	Рильке Райнер Мария
1959	Фрейд	Сартр Жан-Поль
1940	Подростки	Сэлинджер Джером